

Simulación de Entrevista Técnica — Programación Reactiva

Guía de estudio en formato entrevista · Project Reactor, Spring WebFlux y backpressure en el contexto del sistema POS (gateway reactivo)

Guía de estudio en formato entrevista sobre **programación reactiva** (Project Reactor / Spring WebFlux), el modelo que usa el **API Gateway** de este proyecto. Cada sección incluye la **pregunta del entrevistador**, una **respuesta modelo** y **follow-ups**.

Formato sugerido: 45–55 min. Bloques: fundamentos (10'), Reactor Mono/Flux (15'), backpressure (10'), WebFlux (10'), errores y escenarios (10').

0. Warm-up — Reactivo en el proyecto

P ¿Dónde aparece la programación reactiva en este sistema?

R En el **API Gateway**: Spring Cloud Gateway está construido sobre **Spring WebFlux** y **Reactor Netty**, un stack **no bloqueante y reactivo**. Un gateway es I/O-bound (reenvía peticiones a servicios), así que el modelo reactivo le permite manejar **miles de conexiones concurrentes con pocos hilos**, en lugar de un hilo por request como en el modelo servlet bloqueante. Los servicios de negocio (venta, stock, despacho) son MVC tradicional; el reactivo brilla en el borde/proxy.

1. Fundamentos

P1 ¿Qué es programación reactiva y qué problema resuelve?

R Es un paradigma **asíncrono** y **orientado a flujos de datos** con propagación de cambios, no bloqueante y con **backpressure**. Resuelve el problema de escalar I/O: en el modelo bloqueante (un hilo por petición) muchos hilos quedan *esperando* respuestas de red/BD, consumiendo memoria y provocando context-switching. El modelo reactivo libera el hilo mientras espera y lo reutiliza, logrando alta concurrencia con pocos hilos (event loop).

FOLLOW-UP ¿Reactivo siempre es mejor? — No. Para cargas **CPU-bound** o código que llama a librerías bloqueantes (JDBC clásico) el modelo bloqueante es más simple e igual de eficiente. El reactivo gana en **I/O-bound** con alta concurrencia. Todo el stack debe ser no bloqueante para aprovecharlo.

P2 ¿Qué son los 4 principios del Reactive Manifesto?

R Un sistema reactivo es: **Responsive** (responde a tiempo), **Resilient** (se mantiene ante fallos, con aislamiento), **Elastic** (escala con la carga) y **Message-driven** (comunicación asíncrona por mensajes, que habilita las otras tres).

2. Project Reactor: Mono y Flux

P3 ¿Diferencia entre Mono y Flux?

R Ambos son **Publishers** (spec Reactive Streams). **Mono<T>** emite **0 o 1** elemento (p. ej. una respuesta HTTP única). **Flux<T>** emite **0..N** elementos (un stream, p. ej. eventos SSE o filas de una consulta). Son **perezosos**: nada ocurre hasta que hay un *subscribe*.

FOLLOW-UP "Nothing happens until you subscribe" — ¿qué significa? — Un **Mono / Flux** es solo una **receta** (pipeline). No ejecuta nada hasta que alguien se suscribe; en WebFlux el framework hace el subscribe por ti al devolver el publisher del controlador.

P4 ¿Operadores comunes? Muestra un pipeline.

R `map` (transformar), `flatMap` (encadenar operaciones asíncronas), `filter`, `zip` (combinar), `onErrorResume` (fallback).

```
Mono<Producto> producto = webClient.get()
    .uri("/api/stock/{id}", id)
    .retrieve()
    .bodyToMono(Producto.class)
    .timeout(Duration.ofSeconds(2))
    .retryWhen(Retry.backoff(3, Duration.ofMillis(200)))
    .onErrorResume(ex -> Mono.just(Producto.noDisponible(id)));
```

FOLLOW-UP ¿`map` vs `flatMap`? — `map` transforma un valor de forma síncrona ($T \rightarrow U$). `flatMap` transforma en **otro Publisher** asíncrono ($T \rightarrow \text{Mono}<U>$) y lo aplana; se usa para llamadas anidadas no bloqueantes.

3. Backpressure

P5 ¿Qué es backpressure y por qué importa?

R Es el mecanismo por el cual el **consumidor controla el ritmo** al que el productor le envía datos, evitando que un productor rápido **desborde** a un consumidor lento (y agote memoria). En Reactive Streams el subscriber pide `request(n)` elementos; el productor no emite más de lo solicitado. Es la diferencia clave frente a un stream "empujado" ciego.

FOLLOW-UP ¿Estrategias si no se puede frenar al productor? — `onBackpressureBuffer` (encolar), `onBackpressureDrop` (descartar), `onBackpressureLatest` (quedarse con el último). Cada una intercambia memoria por pérdida de datos según el caso.

4. Spring WebFlux

P6 ¿WebFlux vs Spring MVC?

R **MVC**: modelo servlet **bloqueante**, un hilo por request, sobre Tomcat; simple y maduro. **WebFlux**: **no bloqueante** sobre Netty, event loop con pocos hilos, controladores que devuelven `Mono` / `Flux`. WebFlux escala mejor en alta concurrencia I/O-bound (el caso del gateway) pero exige que **todo el pipeline** sea no bloqueante (usar `WebClient`, drivers reactivos como R2DBC o Mongo reactivo).

P7 ¿Qué pasa si haces una llamada bloqueante dentro de WebFlux?

R Bloqueas un hilo del **event loop**, que es un recurso muy escaso (unos pocos hilos). Eso mata la escalabilidad: pocas llamadas bloqueantes concurrentes pueden congelar todo el servidor. Si es inevitable, hay que **desplazar** el trabajo bloqueante a otro scheduler con `subscribeOn(Schedulers.boundedElastic())`.

5. Errores, threading y scheduling

P8 ¿Cómo se manejan errores en un pipeline reactivo?

R Con operadores dedicados: `onErrorReturn` (valor por defecto), `onErrorResume` (pipeline alternativo), `onErrorMap` (traducir excepción), `retry` / `retryWhen` (reintentar con backoff). El error se **propaga por el flujo** como una señal terminal (`onError`), no como una excepción que rompe la pila; hay que tratarlo declarativamente.

P9 ¿Para qué sirven los Schedulers?

R Controlan **en qué hilos** se ejecuta el pipeline. `publishOn` cambia el hilo para los operadores *siguientes*; `subscribeOn` fija el hilo de la *suscripción*. Schedulers típicos: `parallel()` (CPU-bound), `boundedElastic()` (envolver trabajo bloqueante), `immediate()` . Permiten mantener el event loop libre.

6. Diseño abierto / escenarios

P10 ¿Convendría hacer los servicios de negocio reactivos también?

R Depende. Ventajas: mayor concurrencia con menos hilos si son muy I/O-bound (muchas llamadas a Mongo/Kafka). Costos: **curva de aprendizaje**, depuración más difícil (stack traces poco intuitivos), y necesita **drivers reactivos** de punta a punta (Mongo reactivo, R2DBC). Para este POS, con carga moderada, MVC bloqueante en los servicios es una decisión razonable; el reactivo se justifica donde más pega: el **gateway**.

P11 ¿Cómo llamarías a stock y despacho en paralelo y combinarías el resultado?

R Con `Mono.zip` (o `zipWith`): dispara ambas llamadas `WebClient` concurrentemente y combina cuando ambas resuelven, en vez de secuencial. Así la latencia total es la del más lento, no la suma.

```
Mono.zip(stockMono, despachoMono)
    .map(t -> new Resumen(t.getT1(), t.getT2()));
```

Apéndice — Chuleta rápida

Tema	Punto clave
Dónde	API Gateway = Spring Cloud Gateway sobre WebFlux/Reactor Netty (no bloqueante)
Por qué	Alta concurrencia I/O-bound con pocos hilos (event loop) vs hilo-por-request
Mono/Flux	Mono = 0..1; Flux = 0..N; perezosos (nada pasa sin subscribe)
map vs flatMap	map: $T \rightarrow U$ síncrono; flatMap: $T \rightarrow \text{Publisher}$ asíncrono (aplana)
Backpressure	El consumidor pide <code>request(n)</code> ; evita desbordar memoria
WebFlux vs MVC	No bloqueante (Netty) vs bloqueante (Tomcat); todo el pipeline debe ser no bloqueante
Peligro	Bloquear el event loop congela el servidor; usar <code>boundedElastic()</code>
Errores	<code>onErrorResume/Return/Map</code> , <code>retryWhen(backoff)</code> — señal <code>onError</code> declarativa
Paralelo	<code>Mono.zip</code> para combinar llamadas concurrentes